

Learning STARK

Safe Systems Programming for Typed ML Deployment

CHAPTER 03

Values, Types, and Expressions

Learning STARK

Chapter 3 preview: *Values, Types, and Expressions*

Early Access manuscript - July 2026

This chapter documents a language specification under active implementation. Core v1 is a normative draft and has not yet been validated by a conforming compiler.

The design and text are original to the STARK Language Project.

Source manuscript: `book/manuscript/ch03-values-types-expressions.md`

Values, Types, and Expressions

Types are not labels added after a program is written. They are the vocabulary STARK uses to prove what each expression can mean.

Chapter 2 followed one complete Core v1 program. We saw values constructed, borrowed, converted, compared, wrapped in `Result`, and consumed by `match`. This chapter slows down and examines the rules underneath those operations.

Every STARK expression has a type. Sometimes the type is written directly. Sometimes it is inferred from a literal, a function signature, or the surrounding context. Either way, the compiler must resolve it before execution. That fact supports more than editor hints. It determines which operations are legal, which representation a value uses, how ownership behaves, and where runtime checks remain necessary.

Core v1 favors explicit and stable rules. Numeric types do not silently change to make an expression compile. Integer overflow traps in every build configuration. Indexing is bounds checked. A semicolon changes a value-producing expression into a statement. Comparisons cannot be chained as though the language were guessing at mathematical notation.

The result is not a language without runtime failure. Division can still encounter zero, a dynamic index can still be out of bounds, and a cast can still receive a value that does not fit. The design goal is narrower: the same source should have the same type meaning everywhere, and remaining runtime checks should be defined rather than accidental.

Specification status

The rules in this chapter come from the Core v1 normative draft. They describe intended language behavior, not a completed compiler. Examples have been checked against the written grammar and type specification but cannot yet be run through a conforming implementation.

3.1 A value is more than its printed form

The characters `42` do not fully describe a runtime value. The literal could become `Int32`, `Int64`, or another integer type fixed by context or suffix. Those types differ in width, signedness, range, arithmetic behavior at their boundaries, and compatibility with function parameters.

Similarly, the text `"image"` may denote a borrowed string slice rather than an owned, growable `String`. The expression `[1, 2, 3]` carries both an element type and a length. The expression `(1920u32, 1080u32)` is a tuple whose two positions have known types. A reference such as `&size` carries temporary access without ownership.

STARK groups these facts into a static type system. The compiler resolves the type of every expression and checks that operations agree with it before execution.

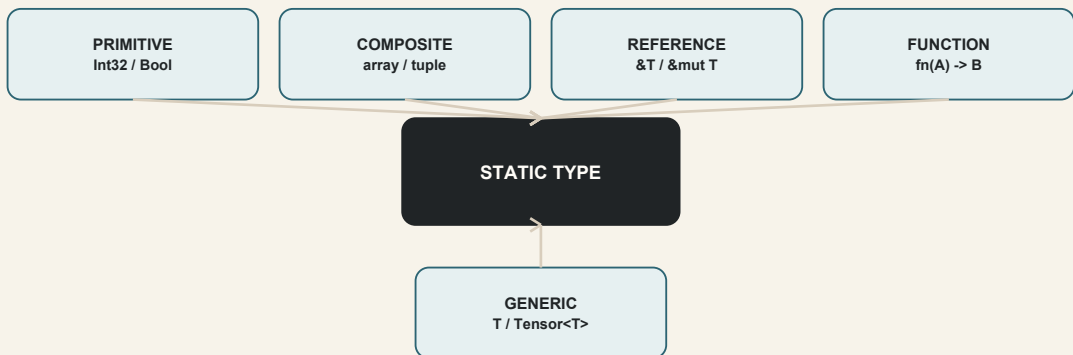


Figure 3-1. Every expression has one static type drawn from a small set of composable families.

The broad families overlap in useful ways:

- **Primitive types** have built-in representations and operations.
- **Composite types** combine other types into arrays, tuples, structs, and enums.
- **Reference types** describe borrowed access to an existing value.
- **Function types** describe named or otherwise non-capturing functions.

- **Generic types** such as `Option<T>`, `Result<T, E>`, and `Vec<T>` are instantiated with other types.

Ownership adds a second question: does an expression copy a small value, move an owned value, or borrow it? Type and ownership are related but not identical. `UInt64` is both a type and a `Copy` value. `String` is a type whose values move by default. `&String` is a reference type whose value represents a live borrow.

This chapter concentrates on value shape and expression typing. Chapters 7 and 8 examine moves and borrows in depth.

3.2 Integer types make width and signedness visible

Core v1 defines eight integer types:

TEXT

Signed		Unsigned	
<code>Int8</code>	<code>-128 .. 127</code>	<code>UInt8</code>	<code>0 .. 255</code>
<code>Int16</code>	<code>-2¹⁵ .. 2¹⁵-1</code>	<code>UInt16</code>	<code>0 .. 2¹⁶-1</code>
<code>Int32</code>	<code>-2³¹ .. 2³¹-1</code>	<code>UInt32</code>	<code>0 .. 2³²-1</code>
<code>Int64</code>	<code>-2⁶³ .. 2⁶³-1</code>	<code>UInt64</code>	<code>0 .. 2⁶⁴-1</code>

The name is part of the contract. An `UInt8` can represent a byte-like value without negative states. An `UInt32` can represent a large image dimension while using a fixed four-byte scalar representation. An `UInt64` provides a wider range for products such as pixel or sample counts.

Signed and unsigned values are not interchangeable merely because the current value is positive:

STARK

```
// Core v1
let signed: Int32 = 42;
let unsigned: UInt32 = 42u32;

// let total = signed + unsigned;
// Error: binary arithmetic requires the same numeric type.
```

The programmer must choose an explicit conversion and therefore confront the destination range:

STARK

```
// Core v1
let total = (signed as UInt32) + unsigned;
```

That cast may trap at runtime if `signed` is negative. The source now shows where the program claims the conversion is valid. In production code, validating the source value before the cast may express the policy more clearly than relying on a trap.

Literal suffixes

An integer suffix fixes the literal type:

STARK

```
let byte = 255u8;           // UInt8
let width = 1920u32;        // UInt32
let count = 4_000_000u64;    // UInt64
let offset = 12i64;         // Int64
```

The suffix uses a compact spelling while the type annotation uses the full Core name. A suffixed literal that does not fit its selected type is a compile-time error:

STARK

```
// let impossible = 256u8;
// Error: 256 is outside the range of UInt8.
```

Underscores may separate digits but only between two valid digits. They are visual punctuation for humans, not part of the value.

Bases

Integer literals may use decimal, hexadecimal, binary, or octal notation:

STARK

```
let decimal = 255u32;
let hex = 0xFFu32;
let binary = 0b1111_1111u32;
let permissions = 0o755u32;
```

All four bindings contain the same numeric value except `permissions`, whose octal digits denote decimal 493. The notation should communicate the domain. Bit masks are often clearer in hexadecimal or binary; quantities are usually clearer in decimal.

3.3 Floating-point values follow IEEE-754

Core v1 provides `Float32` and `Float64`. Unsuffixed floating-point literals default to `Float64`; suffixes select explicitly:

STARK

```
let default_ratio = 16.0 / 9.0;           // Float64
let compact_ratio = 1.777_778f32;        // Float32
let tolerance = 1.0e-6f64;               // Float64
```

Floating-point operations follow IEEE-754 behavior, including NaN and positive or negative infinity. That differs from integer arithmetic, where division by zero and overflow trap.

Floating-point equality deserves care because many decimal fractions have no exact binary representation. The type system can prove that two operands are both `Float64`; it cannot prove that direct equality is the right numerical policy. Domain code commonly compares a difference against a tolerance.

STARK

```
// Core v1
fn approximately_equal(a: Float64, b: Float64, tolerance: Float64) -> Bool {
  abs(a - b) <= tolerance
}
```

The function assumes `abs` is in scope from the standard math surface. All three arguments share one type, so subtraction and comparison require no conversion.

Integer and floating-point conversion

Integer-to-float and float-to-integer conversions use `as`:

STARK

```
let pixels: UInt64 = 2_073_600u64;
let megapixels = (pixels as Float64) / 1_000_000.0;
```

A floating-point to integer cast truncates toward zero and traps if the result does not fit the target type, including NaN and infinity. A conversion to floating point may round because a floating representation cannot exactly encode every large integer.

The explicit cast does not make a conversion lossless. It makes the conversion reviewable.

3.4 Literal defaults are local starting points

An unsuffixed integer literal defaults to `Int32` if it fits and `Int64` otherwise. An unsuffixed floating-point literal defaults to `Float64`.

STARK

```
let attempts = 3;           // Int32
let ratio = 1.5;            // Float64
let enabled = true;         // Bool
```

Use a suffix when another numeric representation is required:

STARK

```
let attempts: UInt8 = 3u8;

fn set_limit(limit: UInt64) { }
set_limit(4_000_000u64);
```

The annotation and parameter check the selected types; they do not request a hidden numeric conversion. The suffix makes the representation agree explicitly.

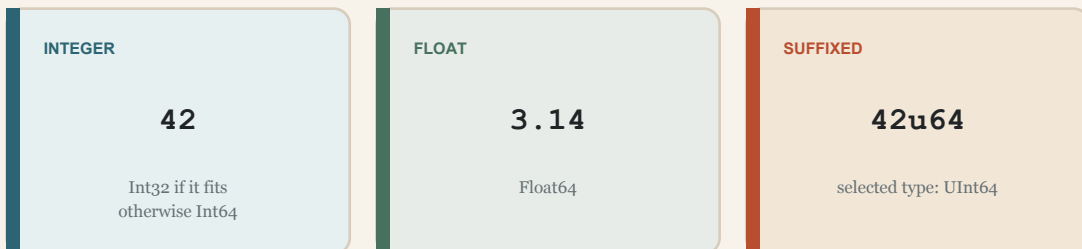


Figure 3-2. Defaults classify unsuffixed literals; suffixes make representation explicit.

Defaults are not a sequence of implicit runtime conversions. They choose the type of an unsuffixed literal. Once the type is selected, ordinary compatibility rules apply.

This distinction prevents a common misunderstanding. In the expression below, the compiler does not promote the left operand until both sides happen to agree:

STARK

```
let small: UInt16 = 10u16;
let large: UInt64 = 20u64;
// let result = small + large; // Error: UInt16 and UInt64 differ.
```

One operand must be explicitly converted:

STARK

```
let result = (small as UInt64) + large;
```

Choose representation at the boundary

Convert values when they enter a calculation or interface, then keep the internal expression in one numeric type. Scattered casts make both correctness and overflow policy harder to review.

3.5 Bool, Char, Unit, and !

Not every primitive is numeric.

Boolean values

`Bool` has two values: `true` and `false`. Logical operators require Boolean operands:

STARK

```
let has_width = width > 0u32;
let has_height = height > 0u32;
let valid = has_width && has_height;
let invalid = !valid;
```

STARK does not treat integers, pointers, collections, or strings as truthy or falsey. An `if` condition must be `Bool`. The expression `width != 0u32` states the conversion from numeric state to logical state explicitly.

`&&` and `||` short-circuit. The right operand is evaluated only when the left operand does not determine the result. They are built-in operations and cannot be overloaded.

Characters

`Char` represents a Unicode scalar value, not an arbitrary byte:

STARK

```
let separator: Char = '/';
let newline: Char = '\n';
let symbol: Char = '\u{03BB}';
```

The last literal uses a Unicode escape. UTF-8 encoding is relevant when a character becomes part of a string or byte sequence, but a `Char` itself represents the scalar value.

Unit

`Unit` represents the absence of a meaningful result. Its value is written `()`:

STARK

```
fn log_ready() {
    println("ready");
}

let result: Unit = log_ready();
```

A function without a return annotation returns `Unit`. A block with no trailing expression also has type `Unit`. `Unit` is a real type with one value, not an untyped hole.

Never

The never type `!` describes an expression that does not produce a value because execution does not continue normally:

STARK

```
fn panic(message: &str) -> !
```

An expression of type `!` can coerce to any expected type. This permits a branch that terminates to coexist with a branch that returns a value:

STARK

```
let size = if available {
    measured_size
} else {
    panic("size unavailable")
};
```

If the `else` branch runs, the binding is never reached. If the binding is reached, the `if` expression produced the type of `measured_size`.

3.6 Borrowed text and owned text are different types

Core v1 distinguishes `str` from `String`.

`str` is an unsized UTF-8 string-slice type and is normally used through a reference, `&str`. A string literal can be used as a borrowed string slice:

STARK

```
let label: &str = "resnet50";
```

`String` is an owned, heap-allocated, growable UTF-8 string:

STARK

```
let owned: String = String::from(label);
```

The difference is about lifecycle and storage, not merely API spelling. The `&str` binding borrows text stored elsewhere. The `String` binding owns its allocation and will release it when its owner leaves scope.

An owned string can provide a borrowed view:

STARK

```
let view: &str = owned.as_str();
println(view);
```

The view cannot outlive `owned`. While the view is live under Core v1's lexical rules, operations requiring an incompatible mutable borrow of `owned` are rejected.

String indexing is deliberately not presented as arbitrary integer indexing. UTF-8 uses a variable number of bytes per scalar value, so byte positions, scalar positions, and user-perceived characters are not interchangeable. The standard-library string APIs expose explicit operations such as `chars`, `bytes`, `substring`, `find`, and `split`, each with its own contract.

Text has at least three useful levels

Use `UInt8` collections for raw bytes, `Char` for a Unicode scalar value, and `String` or `&str` for UTF-8 text. Choosing one level prevents accidental assumptions about indexing and length.

3.7 Arrays carry their length in the type

A fixed-size array type is written `[T; N]`:

STARK

```
let channels: [UInt8; 3] = [12u8, 34u8, 56u8];
```

The element type is `UInt8` and the length is three. `[UInt8; 3]` and `[UInt8; 4]` are different types even though both store bytes.

Local inference preserves the length:

STARK

```
let shape = [1920u32, 1080u32]; // [UInt32; 2]
```

Array literals require compatible element types. A repeat literal creates a fixed number of copies:

STARK

```
let zeros: [UInt8; 16] = [0u8; 16u64];
```

The repeat count must be a compile-time constant expression of unsigned integer type, and the repeated value must implement `Copy`.

Slices borrow a sequence without fixing its length

The type `[T]` is an unsized slice. It cannot be stored directly in a local binding; it appears behind a reference:

STARK

```
let fixed: [Int32; 5] = [1, 2, 3, 4, 5];
let all: &[Int32] = &fixed;
let middle: &[Int32] = &fixed[1..4];
```

`all` borrows the entire array after an array-to-slice reference coercion. `middle` borrows the half-open range containing indices one, two, and three.

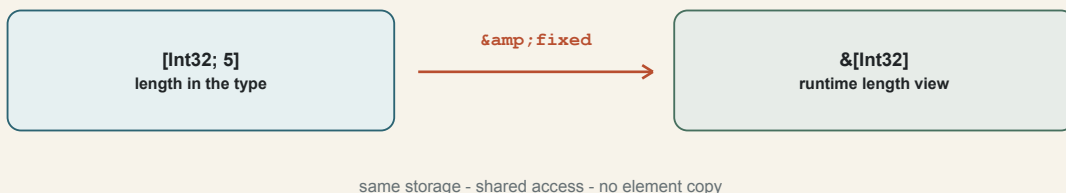


Figure 3-3. Borrowing a fixed array as a slice preserves element type while moving length to runtime.

An array knows its length statically. A slice knows its length at runtime while retaining a statically known element type. A function accepting `&[Int32]` can therefore work with arrays and compatible collection views of many lengths without taking ownership.

Bounds checks remain

Indexing with a statically invalid constant can be rejected at compile time:

STARK

```
let values: [Int32; 3] = [10, 20, 30];  
// let missing = values[5]; // Compile-time error when determinable.
```

A dynamic index requires a runtime bounds check:

STARK

```
let selected = values[index];
```

If `index` is outside the array, execution traps. Static typing proves the element type of a successful access; it does not invent a valid element for an invalid index.

3.8 Tuples group positions; ranges describe traversal

A tuple combines a fixed sequence of potentially different types:

STARK

```
let dimensions: (UInt32, UInt32) = (1920u32, 1080u32);  
let width = dimensions.0;  
let height = dimensions.1;
```

Positions use numeric field access. Unlike a struct, a tuple does not name the roles of its elements. Tuples are useful for small, local groupings and multiple values whose meaning is already obvious. A struct is usually clearer when fields form a durable domain concept.

Parentheses alone do not always create a tuple:

STARK

```
let grouped = (42);           // Int32, grouping only  
let single = (42,);          // (Int32,), one-element tuple  
let empty = ();              // Unit
```

The comma distinguishes a one-element tuple from a grouped expression.

Ranges

The operators `..` and `..=` create range values:

STARK

```
let half_open = 0..10; // 0 through 9
let inclusive = 0..=9; // 0 through 9
```

Integer ranges implement `Iterator` and may drive `for` loops. Ranges also support slicing where the collection contract permits it.

Open-ended ranges such as `start..` or `..=end` are future syntax and are not part of Core v1. Both bounds are present in the current grammar.

Range operators are non-associative. An expression such as `a..=b..=c` is a syntax error rather than a nested range whose meaning must be guessed.

3.9 Local inference solves nearby constraints

Local inference determines `let` binding types from initializers and, where necessary, later uses in the same function body. It never crosses a function boundary because signatures are fully explicit.

STARK

```
fn identity<T>(value: T) -> T {
    value
}

fn example() {
    let count = 42;           // Int32
    let ratio = 3.14;         // Float64
    let values = [1, 2, 3];    // [Int32; 3]
    let same = identity(count); // T = Int32
}
```

The call to `identity` uses argument type to infer the generic parameter. If a generic parameter cannot be inferred from arguments or an expected result, expression-level generic arguments use the `::<...>` form:

STARK

```
let bytes = size_of::UInt64();
```

Inference has limits by design. It does not select conversions, search arbitrary future uses across modules, or guess a type based on which overload seems convenient. When constraints do not determine one type, the compiler reports that an annotation is required.

Annotations check inferred structure

An annotation can constrain a nested expression:

STARK

```
let pair: (UInt64, UInt64) = (1920u64, 1080u64);
```

The initializer already has type `(UInt64, UInt64)`, and the annotation checks that structure. If either literal used a different numeric type, the annotation would not silently convert it.

Expected typing should not be confused with subtyping. Core v1 has a small set of defined coercions, including mutable-reference to shared-reference, array-reference to slice-reference, and never to any type. Numeric types do not form an implicit widening hierarchy.

3.10 Casts mark representation changes

The `as` operator performs explicit numeric casts:

STARK

```
let width: UInt32 = 1920u32;  
let wide: UInt64 = width as UInt64;  
let floating: Float64 = wide as Float64;
```

Cast behavior is defined by category:

- Integer-to-integer preserves the value when it fits and traps otherwise.
- Float-to-float rounds to the nearest representable value.
- Integer-to-float may round when the integer cannot be represented exactly.
- Float-to-integer truncates toward zero and traps for out-of-range results, NaN, or infinity.

A cast is not a reinterpretation of arbitrary bits. It is a value conversion with defined success and failure behavior.

The cast operator binds more tightly than exponentiation and ordinary arithmetic according to the precedence table. Parentheses are still valuable when a conversion defines the conceptual boundary of a calculation:

STARK

```
let megapixels = (pixels as Float64) / 1_000_000.0;
```

Here the grouping tells the reader: first choose the floating representation for `pixels`, then perform floating-point division.

3.11 Precedence is part of expression meaning

STARK has sixteen precedence levels. You rarely need to recite them all, but you must recognize the major grouping rules.

Postfix operations such as calls, indexing, field access, and `?` bind most tightly. Unary operators follow. Casts bind before exponentiation, multiplication before addition, comparisons before logical conjunction and disjunction, ranges near the bottom, and assignment last.

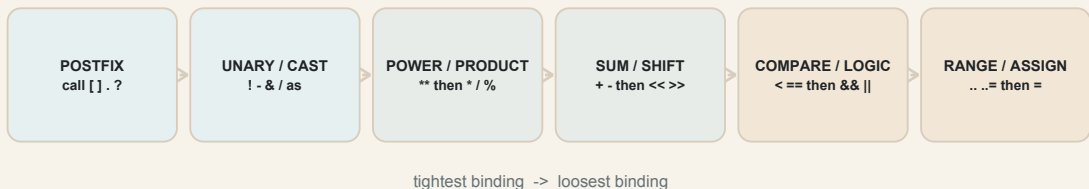


Figure 3-4. Major precedence bands, from tightest binding at left to loosest at right.

Consider:

STARK

```
let total = base + width * height;
```

Multiplication binds first, so the expression means `base + (width * height)`.

STARK

```
let valid = width > 0u32 && height > 0u32;
```

Relational comparisons bind before `&&`, so the expression combines two Boolean results.

Exponentiation and assignment are right-associative; most other binary operators are left-associative. Comparisons and ranges are non-associative:

STARK

```
// a < b < c          // Syntax error
let ordered = a < b && b < c;
```

The expanded Boolean form communicates the intended mathematical relationship and type-checks because both comparisons produce `Bool`.

Parentheses should clarify intent rather than compensate for ignorance of the grammar. A production formatter may remove redundant whitespace, but it should never rewrite grouping in a way that changes the syntax tree.

3.12 Assignment requires a place

The left side of an assignment must denote a storage location, called a **place expression**. Valid places include:

- a variable;
- a field of a place;
- a tuple field of a place;
- an indexed element of a place;
- a dereferenced expression; and
- a parenthesized place.

STARK

```
let mut attempts = 1u32;
attempts = 2u32;
attempts += 1u32;
```

For a mutable struct binding, a field can be a place:

STARK

```
struct Cursor { x: Int32, y: Int32 }

let mut cursor = Cursor { x: 0, y: 0 };
cursor.x = 10;
```

The following expressions produce values but do not identify assignable storage:

STARK

```
// (attempts + 1u32) = 5u32; // Error: arithmetic result is not a place.
// make_cursor() = cursor;   // Error: call result is not a place.
```

Place validity and mutability are separate checks. `cursor.x` is structurally a place, but assignment would still fail if `cursor` were immutable. Assignment through a reference additionally depends on whether the reference is mutable and whether another live borrow conflicts.

Compound assignments such as `+=` preserve the same constraints: the target must be a mutable place, and the operation must be defined for the target and right-hand types.

3.13 Static checking and runtime traps divide the work

Static types reject contradictions available before execution:

- adding `UInt32` to `UInt64` without a cast;
- passing a `String` where `&str` is required without an appropriate borrow or view;
- indexing a statically sized array with a determinably invalid constant;
- assigning through an immutable binding;
- using a non-Boolean `if` condition; and
- returning `Unit` from a function that promises `UInt64`.

Other failures depend on runtime values:

- integer overflow or underflow;
- division or modulo by zero;
- a dynamic index outside an array or slice;
- an integer cast whose value is outside the target range;

- a float-to-integer cast of NaN, infinity, or an out-of-range value; and
- an explicit `panic`.

Core v1 defines these runtime errors as immediate program aborts with a non-zero exit status. There is no panic unwinding, no recovery handler, and no destructor execution for live values during the abort.

COMPILE TIME

- type mismatch
- invalid place
- known bad index

RUNTIME TRAP

- overflow / divide by zero
- dynamic bounds failure
- invalid numeric cast

Figure 3-5. Static rejection handles contradictions; traps guard data-dependent failures.

This policy is strict, but predictable. Integer overflow does not wrap in one build and trap in another. Bounds checking does not disappear merely because optimization is enabled. A cast does not silently truncate an out-of-range integer.

Production programs should still prefer typed, recoverable errors when failure is an expected part of the domain. `Result` is appropriate for malformed input, unavailable files, or validation failures. A trap is the last line of defense for a violated runtime condition, not a replacement for ordinary error design.

****Use types for contracts, `Result` for expected failure, and traps for violated execution rules****

The categories may meet at system boundaries, but keeping them distinct makes both APIs and operational behavior easier to reason about.

3.14 Tracing types through a complete expression

The following Core v1 program derives sample count, aspect ratio, and a support decision from image metadata. It intentionally uses several explicit representations so we can trace them.

STARK

```
// Core v1
struct ImageMetrics {
    width: UInt32,
    height: UInt32,
    channels: UInt8
}

impl ImageMetrics {
    fn sample_count(&self) -> UInt64 {
        let width = self.width as UInt64;
        let height = self.height as UInt64;
        let channels = self.channels as UInt64;
        width * height * channels
    }

    fn aspect_ratio(&self) -> Float64 {
        let width = self.width as Float64;
        let height = self.height as Float64;
        width / height
    }
}
```

STARK

```
// Core v1 - program continued
fn main() {
    let metrics = ImageMetrics {
        width: 1920u32,
        height: 1080u32,
        channels: 3u8
    };

    let samples = metrics.sample_count();
    let ratio = metrics.aspect_ratio();
    let dimensions = (metrics.width, metrics.height);

    let has_rgb_channels = metrics.channels == 3u8;
    let within_limit = samples <= 8_000_000u64;
    let supported = has_rgb_channels && within_limit;

    if supported {
        println("image profile supported");
    } else {
        println("image profile rejected");
    }
}
```

Start with the struct literal. The field declarations require `UInt32`, `UInt32`, and `UInt8`. The suffixed literals agree exactly, so `metrics` is inferred as `ImageMetrics`.

Inside `sample_count`, each field read copies a small integer from a shared borrow of `self`. The three `as UInt64` casts create uniform operands. Multiplication is left-associative, so the final expression groups as `(width * height) * channels`, with type `UInt64` throughout. Either multiplication can trap on overflow.

Inside `aspect_ratio`, both operands become `Float64`. Division therefore uses floating-point semantics. A zero height would produce an IEEE-754 infinity or NaN depending on the numerator rather than an integer division-by-zero trap. A real image-validation policy should reject zero dimensions before this calculation, as Chapter 2 did.

Back in `main`, the method signatures determine `samples: UInt64` and `ratio: Float64`. `ratio` is unused in this minimal program; a conforming compiler may warn about that without rejecting the program.

The tuple `dimensions` is inferred as `(UInt32, UInt32)`. Both field reads copy their values. The tuple owns those copies and carries the positional length in its type.

`metrics.channels == 3u8` compares two `UInt8` values and produces `Bool`. `samples <= 8_000_000u64` compares two `UInt64` values and also produces `Bool`. The `&&` operator therefore receives two Boolean operands and produces `supported: Bool`.

Finally, the `if` condition is Boolean. There is no `else` value to bind; both branches call `println` and have type `Unit`, so the `if` expression has type `Unit`. It is the final expression of `main`, matching `main`'s implicit `Unit` return.

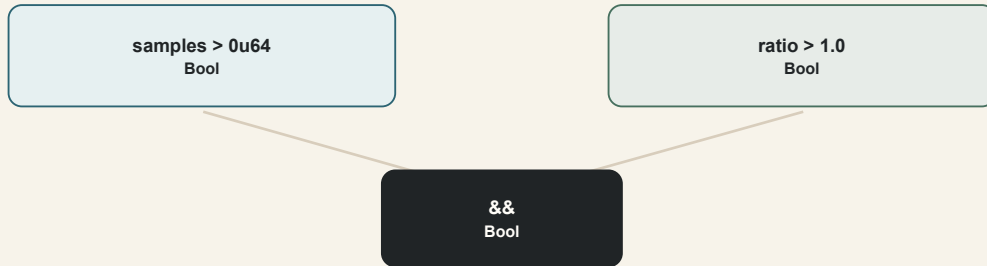


Figure 3-6. Type-checking proceeds from leaves to operators until the whole condition is `Bool`.

The diagram captures a useful checking habit: write down leaf types, apply the rule for one operator or call, then carry the resulting type outward. Large expressions become a series of local obligations rather than one guess.

3.15 A repeatable method for tracing expressions

When an expression is unfamiliar, work from the most tightly bound operations outward:

1. **Resolve names.** Identify each binding, field, function, and associated item.
2. **Type the leaves.** Record literal types, binding types, and function signatures.
3. **Apply postfix operations.** Resolve calls, indexing, fields, methods, and `?`.
4. **Apply unary operations and casts.** Note borrows, dereferences, negation, and representation changes.
5. **Follow precedence.** Group exponentiation, arithmetic, shifts, comparisons, logic, ranges, and assignment in order.
6. **Check operand compatibility.** Arithmetic operands must share a numeric type; logical operands must be Boolean.
7. **Check ownership and place rules.** Determine whether a value is copied, moved, borrowed, or assigned.
8. **Apply the expected type.** Use the binding, return, argument, or branch context to constrain the final result.

For the expression:

STARK

```
let supported = metrics.channels == 3u8 && samples <= 8_000_000u64;
```

the relational and equality operators bind before `&&`. Each comparison is independently checked and produces `Bool`. Only then does logical conjunction combine the results. Writing the intermediate facts makes it obvious why this expression is valid and why `metrics.channels && samples` would not be.

Chapter summary

- Every expression has a statically resolved type, whether written or inferred.
- Integer types make width and signedness explicit; arithmetic operands must share one numeric type.
- Unsuffixed integer literals default to `Int32` or `Int64`, and floating literals default to `Float64` when context does not choose otherwise.
- Numeric casts use `as` and have defined rounding or trapping behavior.
- `Bool` has no truthy conversions, `Char` represents a Unicode scalar value, `Unit` has one value, and `!` represents a computation that never returns.
- `String` owns growable UTF-8 text; `&str` borrows a string slice.
- Array types contain a static length, while borrowed slices support dynamic lengths with a static element type.
- Tuples group fixed positions, and ranges create iterable or slicing values with explicit bounds.
- Local inference solves constraints within a function but does not invent numeric conversions or infer function signatures.
- Precedence and associativity determine expression structure; chained comparisons and chained ranges are rejected.
- Assignment targets must be mutable place expressions.
- Static checking rejects type contradictions, while defined runtime traps handle value-dependent overflow, bounds, division, and casts.

Exercises

1. Determine the type of each binding: `let a = 1;`, `let b = 1u8;`, `let c = [1, 2, 3];`, and `let d = (1u32, 2.0);`.
2. Explain why `let total = 1u16 + 2u64;` is rejected. Provide two valid rewrites and describe how their range policies differ.

3. What are the types and values of `(42)`, `(42,)`, and `()`?
4. Trace the type of `width > 0u32 && height > 0u32` from its leaves to its result.
5. Rewrite a function taking `[UInt8; 3]` so that it can borrow arrays or collection views of any length. What information moves from compile time to runtime?
6. Give one example each of a type error, an expected failure best represented by `Result`, and a Core runtime trap.
7. Why can `(pixels as Float64) / 1_000_000.0` lose precision even though the cast is explicit?
8. Which of these are place expressions: `cursor.x`, `make_cursor()`, `values[index]`, `a + b`, `*pointer`, and `(cursor.x)`?
9. Add zero-height validation to `aspect_ratio`. Would you represent failure with `Option`, `Result`, or a trap? Defend the interface choice.

Source notes

1. Primitive and composite types, inference, coercions, operator typing, numeric semantics, and safety guarantees follow [STARKLANG/docs/spec/03-Type-System.md](https://starklang.dev/docs/spec/03-Type-System.md).
2. Literal syntax, suffixes, bases, escapes, operators, and token rules follow [STARKLANG/docs/spec/01-Lexical-Grammar.md](https://starklang.dev/docs/spec/01-Lexical-Grammar.md).
3. Expression grammar, precedence, associativity, places, array and tuple literals, ranges, and block values follow [STARKLANG/docs/spec/02-Syntax-Grammar.md](https://starklang.dev/docs/spec/02-Syntax-Grammar.md).
4. Bounds analysis, mutability checks, type-error categories, and runtime abort semantics follow [STARKLANG/docs/spec/04-Semantic-Analysis.md](https://starklang.dev/docs/spec/04-Semantic-Analysis.md).
5. String, slice, iterator, math, memory, and formatting APIs follow [STARKLANG/docs/spec/06-Standard-Library.md](https://starklang.dev/docs/spec/06-Standard-Library.md).